BCC1244 - GAME PROGRAMMING





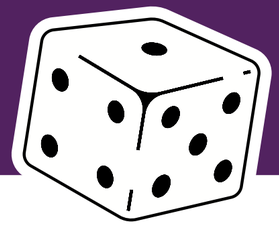
OBJECT-ORIENTED PROGRAMMING IN GAME DEVELOPMENT

START









WHAT IS OBJECT-ORIENTED PROGRAMMING (OOP)?

WHAT IS OBJECT-ORIENTED PROGRAMMING (OOP)?

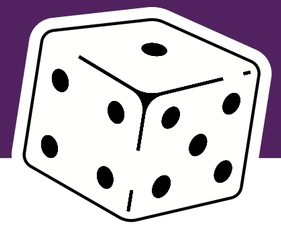
Definition

OOP is a programming paradigm that organizes software design around "objects" rather than functions and logic. An object is a data structure that contains both data (attributes) and behavior (methods).

Why OOP in Games?

Games are complex systems with many interacting entities (e.g., players, enemies, items). OOP helps manage this complexity by encapsulating behavior and data into reusable components.





CORE PRINCIPLES OF OOP

CORE PRINCIPLES OF OOP

Class

A blueprint or template for creating objects. It defines the properties (attributes) and behaviors (methods) that the objects will have.

Example:

A Player class might have attributes like health, speed, and methods like move(), attack().

CORE PRINCIPLES OF OOP

Object

An instance of a class. Each object has its own state (values for attributes) but shares the same behavior (methods).

Example:

Two Player objects, player1 and player2, can have different health values but both can move() and attack().

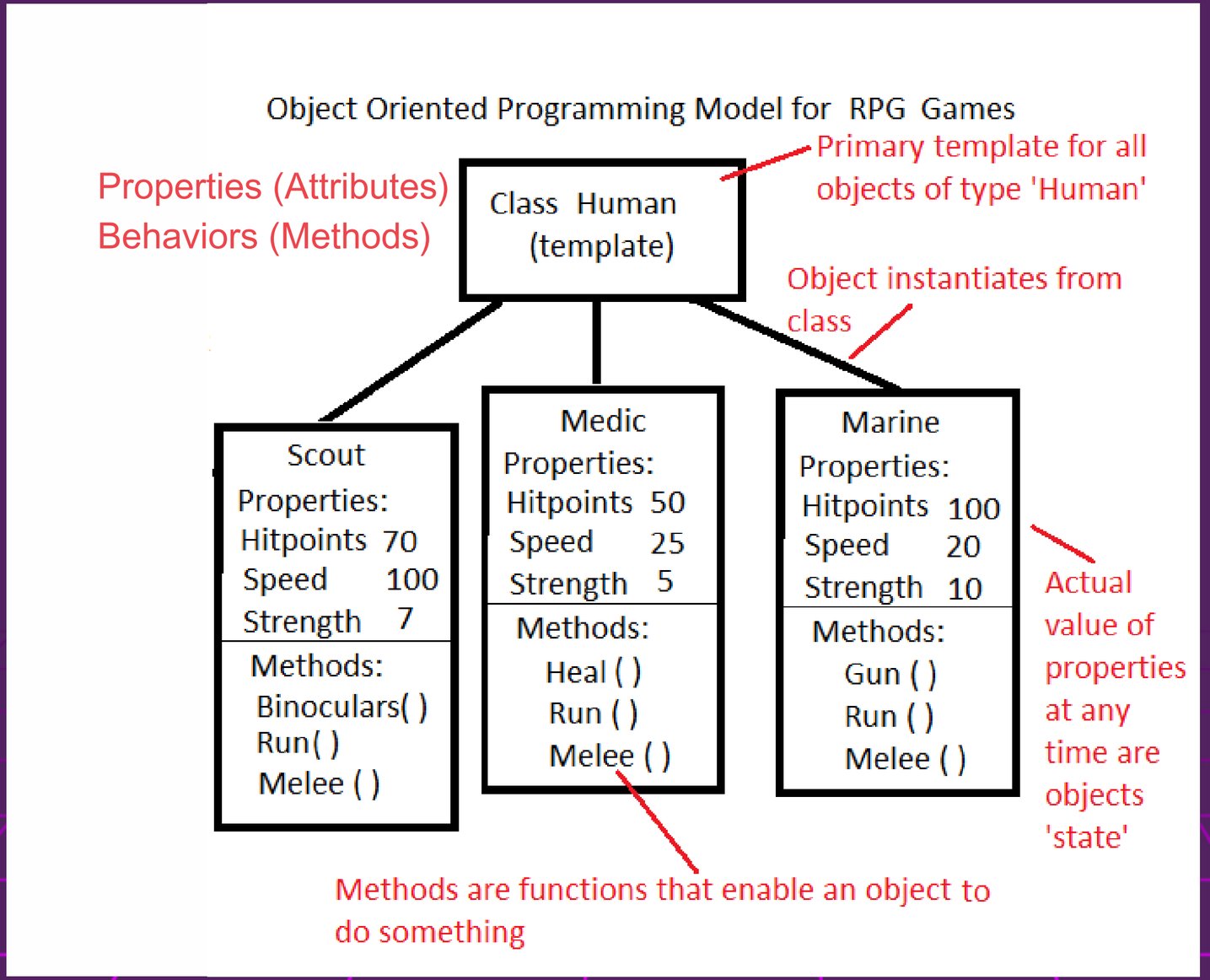
CORE PRINCIPLES OF OOP

Game Example

A Character class could be used to create multiple characters in a game, each with unique attributes like name, health, and weapon.



CORE PRINCIPLES OF OOP



CORE PRINCIPLES OF OOP

Inheritance

A mechanism where a new class (child class) derives properties and behaviors from an existing class (parent class).

Purpose:

Promotes code reuse and hierarchical organization.

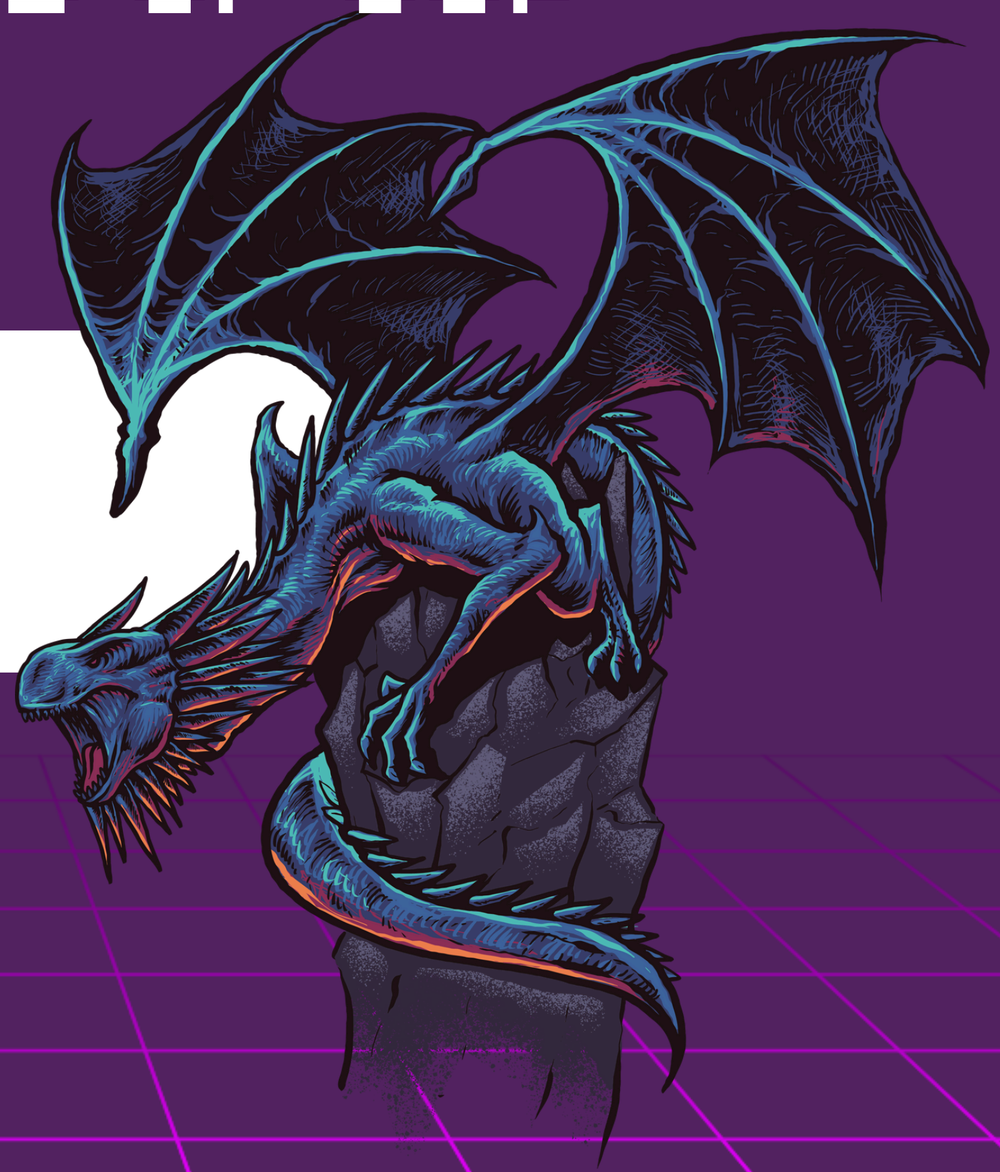
Example:

A Enemy class can inherit from a Character class, adding new attributes like damage and methods like chasePlayer().

CORE PRINCIPLES OF OOP

Game Example

A BossEnemy class could inherit from Enemy, adding special abilities like fireBreath().









CORE PRINCIPLES OF OOP

Polymorphism

The ability of objects to take on multiple forms. It allows methods to behave differently based on the object that calls them.

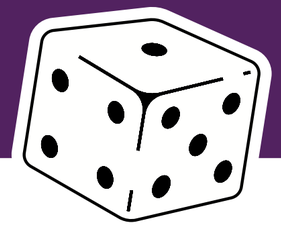
Purpose:

Enables flexibility and extensibility in code.

Example:

A render() method could be defined in a GameObject class, but each subclass (e.g., Player, Enemy) implements it differently to render their unique visuals.





APPLYING OOP IN GAME DEVELOPMENT

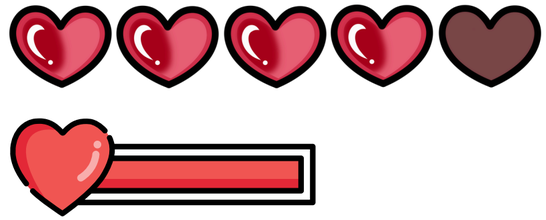
APPLYING OOP IN GAME DEVELOPMENT

Game Entities as Objects

Every entity in a game (e.g., players, enemies, items, obstacles) can be modeled as an object.

Example:

- *Player*: *Attributes* like *health*, *inventory*; *methods* like *jump()*, *shoot()*.
- *Enemy*: *Attributes* like *damage*, *speed*; *methods* like *attack()*, *patrol()*.
- *Item*: *Attributes* like *name*, *effect*; *methods* like *use()*, *pickUp()*.





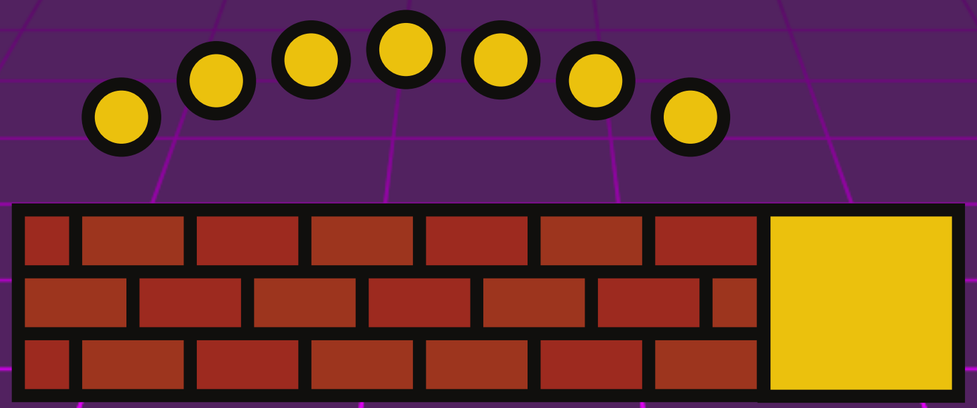
APPLYING OOP IN GAME DEVELOPMENT

Game Systems and OOP

Systems like physics, AI, and inventory management can be designed using OOP principles.

Example:

- *A PhysicsEngine class could handle collision detection and movement for all game objects.*
- *An Inventory class could manage items collected by the player.*







THANK YOU FOR PLAYING!

